

第5次作业总结

题4

题目

Extend the array representation of a complete binary tree to the case of complete trees whose degree is $d, d > 1$. Develop formulas for the parent and children of the node stored in position i of the array.

参考答案

可参考如下解答：

设节点的标号从1开始。

- 子节点公式推导：

度为 d ，说明每个节点都有 d 个子节点，因此可以得出：

- 根节点 (1) 有 d 个子节点： $2, 3, \dots, d+1$ 。
- 节点2的 d 个子节点： $d+2, d+3, \dots, 2d+1$ 。
- 节点3的 d 个子节点： $2d+2, 2d+3, \dots, 3d+1$ 。

通过归纳可以知道：节点 i 的子节点从 $d(i-1)+2$ 开始，到 $di+1$ 结束。

对于第 $i+1$ 个节点，代入上述公式进行验证，则节点 $i+1$ 的第一个节点为 $d(i+1-1)+2=di+2$ ，与第 i 个节点的最后一个子节点可以衔接。

因此节点 i 的第 k 个子节点 ($k=1,2,\dots,d$) 公式为：子节点 = $d(i+1)+k+1$ 。即第一个子节点为 $d(i-1)+2$ ，最后一个子节点为 $d(i-1)+d+1=di+1$ 。

- 父节点公式推导：

假设节点 i 的父节点是 p 。

根据子节点公式， p 的第一个子节点是： $d(p-1)+2$ ；最后一个子节点是： $dp+1$ 。

因此，节点 i 必须在这两个值之间： $d(p-1)+2 \leq i \leq dp+1$

解这个不等式得到： $\frac{i-1}{d} \leq p \leq \frac{i+d-2}{d}$ ，且上下界差值为： $\frac{i+d-2}{d} - \frac{i-1}{d} = 1 - \frac{1}{d} < 1$ 。

由于 p 是整数，取下界或上界都可以得到 p ，因此，节点 i 的父节点 $p = \lfloor \frac{i+d-2}{d} \rfloor$ 或 $p = \lceil \frac{i-1}{d} \rceil$ ，两种表示方法都可以。

题1

题目

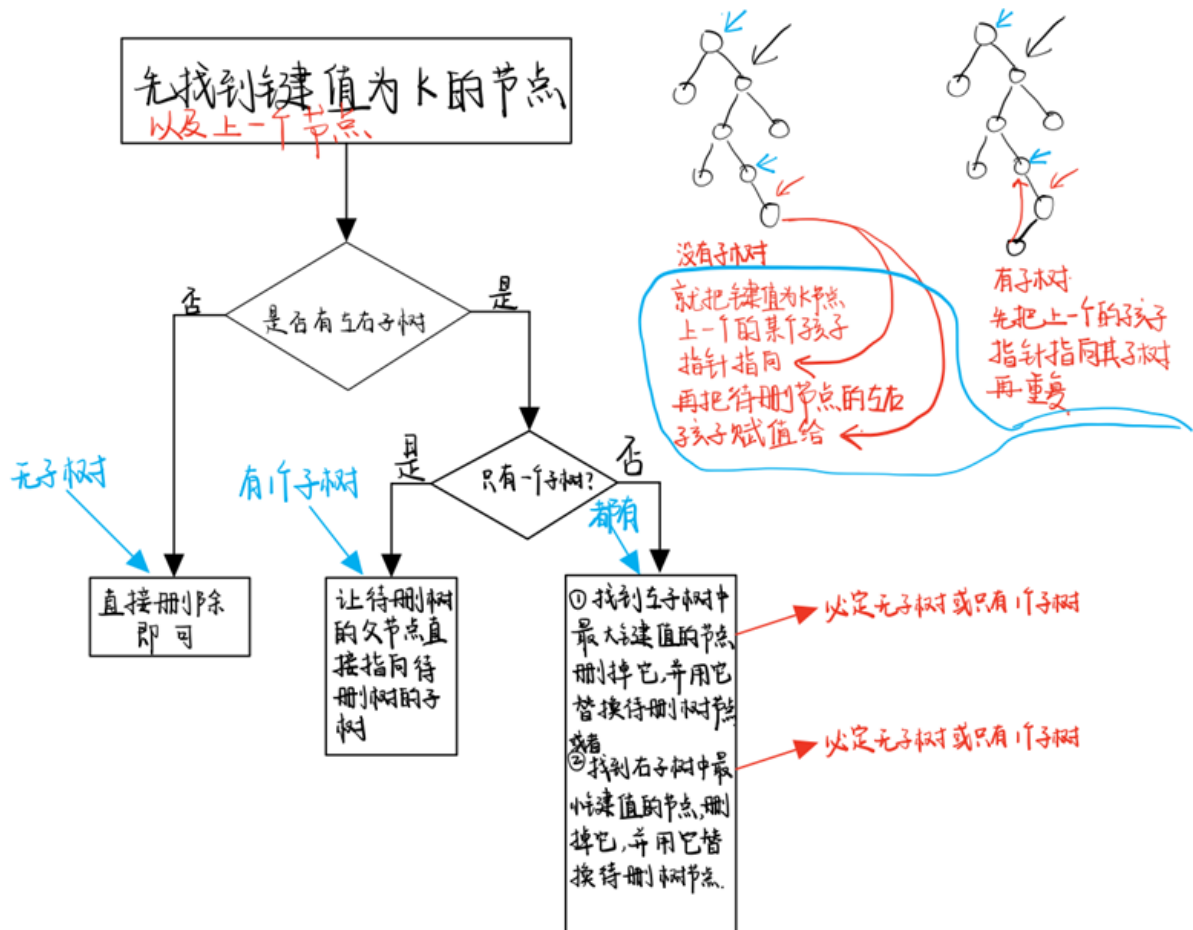
Write a C++ function to delete the pair with key k from a binary search tree. What is the time complexity of your function?

参考答案

二叉查找树中删除键值为k的元素

可参考如下解答：

↓用于维持二叉查找树的结构
(前序遍历结果从小到大)



```
1  template<class K, class E>
2  pair<K, E> *BST<K, E>::Delete(K &k) {
3      // 1. 查找待删除节点 currentNode 及其父节点 Parent
4      TreeNode<pair<K, E>>* currentNode = root;
5      TreeNode<pair<K, E>>* parent = nullptr;
6      while (currentNode != nullptr) {
7          if (k < currentNode->data.first) {
8              // 键值更小, 向左子树查找
9              parent = currentNode;
10             currentNode = currentNode->leftNode;
```

```

11         } else if (k > currentNode->data.first) {
12             // 键值更大，向右子树查找
13             parent = currentNode;
14             currentNode = currentNode->rightNode;
15         } else {
16             // 找到目标节点，退出查找循环
17             break;
18         }
19     }
20     // 未找到目标节点，返回空指针
21     if (currentNode == nullptr) {
22         return nullptr;
23     }
24
25     // 保存要返回的数据
26     pair<K, E>* result = new pair<K, E>(currentNode->data);
27
28     // 2. 分情况处理节点删除（根据子节点数量）
29     // 情况1：待删除节点是叶子节点（无左右子树）
30     if (currentNode->leftNode == nullptr && currentNode->rightNode ==
nullptr) {
31         if (parent != nullptr) {
32             // 更新父节点的子节点指针（置空）
33             if (parent->leftNode == currentNode) parent->leftNode =
nullptr;
34             else parent->rightNode = nullptr;
35         } else root = nullptr; // 特殊情况：删除的是根节点且根节点是叶子
36         delete currentNode;
37     }
38     // 情况2：待删除节点只有右子树（左子树为空）
39     else if (currentNode->leftNode == nullptr) {
40         TreeNode<pair<K, E>>* child = currentNode->rightNode;
41         if (parent == nullptr) {
42             root = child;
43         } else if (parent->leftNode == currentNode) {
44             parent->leftNode = child;
45         } else {
46             parent->rightNode = child;
47         }
48         delete currentNode;
49     }
50     // 情况3：待删除节点只有左子树（右子树为空）
51     else if (currentNode->rightNode == nullptr) {
52         TreeNode<pair<K, E>>* child = currentNode->leftNode;
53         if (parent == nullptr) {
54             root = child;

```

```

55     } else if (parent->leftNode == currentNode) {
56         parent->leftNode = child;
57     } else {
58         parent->rightNode = child;
59     }
60     delete currentNode;
61 }
62 // 情况4: 待删除节点有左右子树 (选左子树最右节点作为替换)
63 else {
64     // 寻找左子树的最右节点 (前驱节点) 作为替换节点
65     TreeNode<pair<K, E>>* insteadNode = currentNode->leftNode;
66     TreeNode<pair<K, E>>* insteadParent = currentNode;
67     while (insteadNode->rightNode != nullptr) {
68         insteadParent = insteadNode;
69         insteadNode = insteadNode->rightNode;
70     }
71     // 更新父节点指向替换节点
72     if (parent != nullptr) {
73         if (parent->leftNode == currentNode) parent->leftNode =
insteadNode;
74         else parent->rightNode = insteadNode;
75     } else {
76         // 替换根节点
77         root = insteadNode;
78     }
79     // 处理替换节点的子节点 (若有左子树)
80     if (insteadNode != currentNode->leftNode) {
81         insteadParent->rightNode = insteadNode->leftNode;
82         insteadNode->leftNode = currentNode->leftNode;
83     }
84     // 替换节点继承待删除节点的右子树
85     insteadNode->rightNode = currentNode->rightNode;
86     delete currentNode;
87 }
88 // 返回被删除的键值对
89 return result;
90 }

```

由于寻找插入位置和要替换的结点都只在二叉树每层找一个，因此时间复杂度为 $O(h)$ (h 为二叉树高度)

题2

题目

Do the inorder and postorder sequences of a binary tree uniquely define the binary tree? Prove your answer.

参考答案

可参考如下解答：

设树中总共有 n 个节点，它的后序遍历为 $a_1, a_2, \dots, a_n$ ，中序遍历为 $b_1, b_2, \dots, b_n$ 。

当 $n = 1$ ，树只有一个节点，显然能够被确定。(1)

设当 $n \leq k$ 时，树能够被后序遍历和中序遍历确定，(2)

当 $n = k + 1$ 时， a_n 为树的根节点，

若 $a_n = b_i (1 \leq i \leq n)$ ， a_n 的左子树总共 $i - 1 (i - 1 \leq k)$ 个节点，后序遍历为 $a_1, \dots, a_{i-1}$ ，中序遍历为 $b_1, \dots, b_{i-1}$ ，由(2)可知，左子树能够被确定； a_n 的右子树总共 $n - i (n - i \leq k)$ 个节点，后序遍历为 $a_i, \dots, a_{n-1}$ ，中序遍历为 $b_{i+1}, \dots, b_n$ ，由(2)可知，右子树能够被确定。因此整棵树都能被确定。

综上，由数学归纳法可知，树能够被它的后序遍历和中序遍历确定。

题4

题目

Do the inorder and level-order sequences of a binary tree uniquely define the binary tree? Prove your answer.

参考答案

可参考如下解答：

设树中总共有 n 个节点，它的层次遍历为 $a_1, a_2, \dots, a_n$ ，中序遍历为 $b_1, b_2, \dots, b_n$ 。

当 $n = 1$ ，树只有一个节点，显然能够被确定。(1)

设当 $n \leq k$ 时，树能够层次遍历和中序遍历确定，(2)

当 $n = k + 1$ 时， a_n 为树的根节点，

若 $a_n = b_i (1 \leq i \leq n)$, a_n 的左子树总共 $i - 1 (i - 1 \leq k)$ 个节点, 层次遍历为 $a_2, \dots, a_n$ 中 $b_1, \dots, b_{i-1}$ 按顺序排列的序列, 中序遍历为 $b_1, \dots, b_{i-1}$, 由(2)可知, 左子树能够被确定;
 a_1 的右子树总共 $n - i (n - i \leq k)$ 个节点, 层次遍历为 $a_2, \dots, a_n$ 中 $b_{i+1}, \dots, b_n$ 按顺序排列的序列, 中序遍历为 $b_{i+1}, \dots, b_n$, 由(2)可知, 右子树能够被确定。因此整棵树都能被确定。

综上, 由数学归纳法可知, 树能够被它的层次遍历和中序遍历确定。